

# L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub> 用于自定义宏包和类的指导书

Copyright © 1995–2006 The L<sup>A</sup>T<sub>E</sub>X Project

All rights reserved.<sup>†</sup>

2006年2月15日

2024 年 3 月 28 日

## 目录

|          |                                                                    |          |
|----------|--------------------------------------------------------------------|----------|
| <b>1</b> | <b>简介</b>                                                          | <b>3</b> |
| 1.1      | 为L <sup>A</sup> T <sub>E</sub> X 2 <sub>ε</sub> 编写文档类和宏包 . . . . . | 3        |
| 1.2      | 总括 . . . . .                                                       | 3        |
| 1.3      | 更多信息 . . . . .                                                     | 4        |
| 1.4      | 标准类的政策 . . . . .                                                   | 5        |
| <b>2</b> | <b>编写类和宏包</b>                                                      | <b>5</b> |
| 2.1      | 旧版本与版本升级 . . . . .                                                 | 5        |
| 2.2      | “docstrip”和“doc”的使用 . . . . .                                      | 6        |
| 2.3      | 是类还是包? . . . . .                                                   | 6        |
| 2.4      | 命令名称 . . . . .                                                     | 7        |
| 2.5      | 盒子命令和颜色 . . . . .                                                  | 7        |
| 2.6      | 定义文本和数学字符 . . . . .                                                | 8        |
| 2.7      | 命令风格 . . . . .                                                     | 8        |

---

\*此文件可在L<sup>A</sup>T<sub>E</sub>X项目公共许可证的条件下进行分发和修改，可以选择使用此许可证的1.3c版本或（根据您的选择）任何后续版本。有关完整详情，请参阅源文件`clsguide.tex`。

<sup>†</sup>翻译者：贺子旭，翻译者所属单位：华中科技大学，翻译完成日期：2024 年 3 月 28 日

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>3</b> | <b>模板类与宏包的结构</b>                                  | <b>11</b> |
| 3.1      | 标识                                                | 12        |
| 3.2      | 使用模板类与宏包                                          | 13        |
| 3.3      | 选项参数声明                                            | 14        |
| 3.4      | 最小模板类                                             | 16        |
| 3.5      | 示例：本地信件模板类                                        | 17        |
| 3.6      | 示例：新闻简报模板类                                        | 17        |
| <b>4</b> | <b>书写模板类与宏包的命令</b>                                | <b>19</b> |
| 4.1      | 标记识别                                              | 19        |
| 4.2      | 加载文件                                              | 20        |
| 4.3      | 选项参数声明                                            | 21        |
| 4.4      | 选项参数代码内的命令                                        | 22        |
| 4.5      | 选项参数传递                                            | 23        |
| 4.6      | 延迟代码                                              | 24        |
| 4.7      | 选项参数处理                                            | 26        |
| 4.8      | 安全的文件操作                                           | 28        |
| 4.9      | 报错与其他代码                                           | 29        |
| 4.10     | 定义命令                                              | 30        |
| 4.11     | 传递参数                                              | 32        |
| <b>5</b> | <b>杂项命令以及其他</b>                                   | <b>32</b> |
| 5.1      | 版面参数                                              | 32        |
| 5.2      | 大小写转换                                             | 32        |
| 5.3      | “book”模板类中的“openany”选项参数                          | 33        |
| 5.4      | 更好的用户自定义数学显示环境                                    | 33        |
| 5.5      | 标准化间距                                             | 34        |
| <b>6</b> | <b>升级 L<sup>A</sup>T<sub>E</sub>X 2.09 模板类与宏包</b> | <b>35</b> |
| 6.1      | 请先做测试!                                            | 35        |
| 6.2      | 故障排除                                              | 35        |
| 6.3      | 适应兼容模式                                            | 36        |
| 6.4      | 字体命令                                              | 36        |
| 6.5      | 弃用命令                                              | 38        |

# 1 简介

本文档是关于为 $\text{\LaTeX}$ 编写文档类和宏包的介绍，特别关注将现有的 $\text{\LaTeX}$  2.09宏包升级为 $\text{\LaTeX}$  2 $\epsilon$ ，其中 $\text{\LaTeX}$  2 $\epsilon$ 可以参考 Johannes Braams 在 TUGboat 15.3 上发表的文章。

## 1.1 为 $\text{\LaTeX}$ 2 $\epsilon$ 编写文档类和宏包

$\text{\LaTeX}$ 是一个精细调控的文档准备系统，该系统使文档作者能够专注于文本的内容，而无需过于担心其格式。例如表示章节时作者可以使用 `\chapter{title}`，而不需要考虑选择 18pt 加粗字体作为其格式。

文档类，即 *document class*，是包含如何将逻辑结构转换为格式信息的文件（如“`\chapter`”转化为“18pt 加粗左对齐”），而独立于文档类的功能（如颜色或插入的图形）则被包含在需要引入的宏包（*package*）中。

$\text{\LaTeX}$  2.09和 $\text{\LaTeX}$  2 $\epsilon$ 之间最显著的区别之一在于编写宏包以及文档类时使用的命令。在 $\text{\LaTeX}$  2.09中，几乎没有支持用于编写 `.sty` 文件，因此编写者必须从底层命令写起。而 $\text{\LaTeX}$  2 $\epsilon$ 提供了用于构建宏包的高级命令，使得在类和宏包的堆叠嵌套之上构建类和宏包也变得更加容易。例如，本地技术报告类 `cetechr`（一般用于化学工程专业用）是基于 `article` 文档类型编写的。

## 1.2 总括

本文档简要描述了如何为 $\text{\LaTeX}$ 编写文档类和宏包方法。该文档不涵盖编写宏包所需的所有命令，若有需要可以在 $\text{\LaTeX}$ : *A Document Preparation System*或 $\text{\LaTeX}$  *Companion*中查找相关命令。本文档着重介绍 $\text{\LaTeX}$  2 $\epsilon$ 中新增构建类和宏包的新增命令。

**第 2.7 节** 将提供一些关于编写类和宏包的一般性建议。该章节包含类和宏包之间的区别、命令的命名约定，`doc` 和 `docstrip` 的使用，以及 $\text{\TeX}$ 的原始文件和框命令同  $\text{\LaTeX}$  的交互方式。该章节还将提供一些有关一般 $\text{\LaTeX}$ 风格的提示。

**第 3 节** 将提供类与宏包的结构描述，包括在其他类和宏包之上构建类和宏包的方法，选项与命令的声明，并提供一些一些类示例。

**第 4 节** 将列出新的类和宏包命令。

**第 6 节** 将给出如何将现有的 $\text{\LaTeX}$  2.09类和宏包升级到 $\text{\LaTeX}$  2 $\epsilon$ 的详细建议。

### 1.3 更多信息

要了解有关 $\text{\LaTeX}$ 的一般介绍，包括 $\text{\LaTeX}$  2 $\epsilon$ 的新功能，您可以参考 Leslie Lamport 的《 *$\text{\LaTeX}$ : A Document Preparation System*》[2]。

关于 $\text{\LaTeX}$ 的新功能的更详细描述，包括对200多个宏包和近1000个可运行示例的概述，可以在 Frank Mittelbach 和 Michel Goossens 共同著作的《*The  $\text{\LaTeX}$  Companion* 第二版》[3] 中找到。

$\text{\LaTeX}$ 系统基于 $\text{\TeX}$ 建立，有关 $\text{\TeX}$ 的内容可以参考 Donald E. Knuth 在《*The  $\text{\TeX}$ book*》[1] 中对其进行的描述。

每个 $\text{\LaTeX}$ 副本都附带了一些文档文件。每过六个月， $\text{\LaTeX}$ 发布版本都会附带一份《 *$\text{\LaTeX}$  News*》，其内容包含在文件`ltnews*.tex`中。作者指南《 *$\text{\LaTeX}$  2 $\epsilon$  for Authors*》描述了新的 $\text{\LaTeX}$ 文档功能，位于`usrguide.tex`中。说明书《 *$\text{\LaTeX}$  2 $\epsilon$  Font Selection*》为类和宏包的编写者描述了 $\text{\LaTeX}$ 字体选择方案，位于 `fontguide.tex`中。配置 $\text{\LaTeX}$ 的方法在指南《*Configuration options for  $\text{\LaTeX}$  2 $\epsilon$* 》中有所介绍，位于`cfgguide.tex`中，而有关修改 $\text{\LaTeX}$ 的政策背后的哲学在《*Modifying  $\text{\LaTeX}$* 》中有所描述，则位于`modguide.tex`中。

源代码文档（用于通过`latex.ltx`生成内核格式的文件）现在以“*The  $\text{\LaTeX}$  2 $\epsilon$  Sources*”的形式提供。这个非常庞大的文档还包括 $\text{\LaTeX}$ 命令的索引。可以使用来自`base`目录的 $\text{\LaTeX}$ 文件—`source2e.tex`—，使用此目录中的源文件和类文件 `ltxdoc.cls`进行排版。

有关 $\text{\TeX}$ 和 $\text{\LaTeX}$ 的更多信息，请联系您所在地的 $\text{\TeX}$ 用户组，或国际 $\text{\TeX}$ 用户组。有关地址和其他详细信息，请参阅：

<http://www.tug.org/lugs.html>

## 1.4 标准类的政策

我们收到的关于标准类的问题报告中，许多并非涉及错误，而是或多或少地提出这些类中体现的设计决策“不够优化”，并要求我们对其进行修改。

以下是我们不应对这些文件进行更改的几个主要原因。

- 无论多么错误，当前的行为完全满足设计这些类时预期的作用。
- 更改“标准类”的这些方面并不是一个好的做法，因为这牵扯到依赖该类运作的项目。

因此，我们决定不考虑进行这些修改，甚至也不愿意花时间为这一决定辩解。这并不意味着我们不同意这些类的设计存在许多不足，但在面对相当多更为紧迫的任务时，不断解释 $\text{\LaTeX}$ 标准类不修改的原因这项任务只能靠边站。

当然，我们欢迎制作更为优秀的类，或者用于增强这些类的宏包。因此，当你考虑到这样的不足时，我们希望您首先想到的是“我能做什么来改善这个问题呢？”

类似的考量同样适用于实施设计决策的内核的那些部分，其中许多决策应该留给类文件，但在当前系统中并非如此。我们意识到，在这种情况下，你很难自己纠正问题，但也同样是情况是在内核中进行这样的更改可能对我们来说是一个重大的项目；因此，这样的增强将不得不挪到 $\text{\LaTeX}$ 3大更新来做。

## 2 编写类和宏包

本节涵盖了与编写 $\text{\LaTeX}$ 类和宏包相关的一些一般性要点。

### 2.1 旧版本与版本升级

如果您要升级现有的 $\text{\LaTeX}$  2.09 样式文件，则建议冻结 2.09 版本并不再维护它。根据 1990 年代初存在的各种 $\text{\LaTeX}$  方言的经验，维护不同版本的 $\text{\LaTeX}$  包几乎是不可能的。

当然，有一些机构可能需要在一段时间内同时维护两个版本，但对于由个人支持的那些包和类来说，这并非必需：它们应该仅支持新的标准 L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub>，而不是旧版本的 L<sup>A</sup>T<sub>E</sub>X。

## 2.2 “docstrip” 和 “doc” 的使用

如果您要编写一个大型的 L<sup>A</sup>T<sub>E</sub>X 类或包，则应考虑使用连同 L<sup>A</sup>T<sub>E</sub>X 一并提供的 doc 软件。使用 doc 编写的 L<sup>A</sup>T<sub>E</sub>X 类和包可以通过两种方式进行处理：一是可以通过 L<sup>A</sup>T<sub>E</sub>X 运行，以生成文档；二是使用 docstrip 进行处理，以生成 .cls 或 .sty 文件。

doc 软件可以自动生成定义的索引、命令使用的索引和更改日志列表。对于维护和记录大型 T<sub>E</sub>X 源非常有用。

L<sup>A</sup>T<sub>E</sub>X 内核本身以及标准类等的文档源都是 doc 文档；它们位于分发的 .dtx 文件中。实际上，可以通过在 source2e.tex 上运行 L<sup>A</sup>T<sub>E</sub>X 来排版内核的源代码，包括索引。排版这些文档所使用的是类文件 ltxdoc.cls。

有关 doc 和 docstrip 的更多信息，请参阅文件《docstrip.dtx》、《doc.dtx》和《The L<sup>A</sup>T<sub>E</sub>X Companion》。有关使用示例，请查看 .dtx 文件。

## 2.3 是类还是包？

在将一些新的 L<sup>A</sup>T<sub>E</sub>X 命令放入文件时，首先要做的是决定它是一个文档类（*document class*）还是包（*package*）。经验法则是：

如果这些命令可以与所有文档类一起使用，则将它们制作成一个包；如果这些命令不能与所有文档类一起使用，则制作成一个类。

有两种主要类型的类：像 `article`、`report` 或 `letter` 这样的独立类；以及那些是其他类的扩展或变体的类—例如，基于 `article` 文档类构建的 `proc` 文档类。

因此，一个公司可以创建一个用于打印带有自己标头信纸的信件的本地 `ownlet` 类。这样的类将基于现有的 `letter` 类创建，但不能与任何其他文

档类一起使用，因此该公司需要创建的是 `ownlet.cls` 而不是 `ownlet.sty`。

相反，`graphics` 宏包提供了将图像包含到  $\text{\LaTeX}$  文档中的命令。由于这些命令可以与任何文档类一起使用，因此我们有 `graphics.sty` 而不是 `graphics.cls`。

## 2.4 命令名称

$\text{\LaTeX}$  有三种类型的命令。

首先是写作命令，例如 `\section`、`\emph` 和 `\times`。其中大多数写作命令都有短名称，而且都是小写的。

其次是类命令与包命令，这些命令名称大多数都较长而且混合大小写的名称，如以下所示：

```
\InputIfFileExists \RequirePackage \PassOptionsToClass
```

最后，还有  $\text{\LaTeX}$  实现中使用的内部命令，如 `\@tempcnta`、`\@ifnextchar` 和 `\@eha`。这些命令的大多数名称中包含 `@`，这意味着它们不能在文档中使用，只能在类和包文件中使用。

不幸的是，由于历史原因，这些命令之间的区别经常变得模糊。例如，`\hbox` 是一个内部命令，只应在  $\text{\LaTeX}$  核心中使用，但 `\m@ne` 是常量  $-1$ ，本可以是 `\MinusOne`。

然而，这个经验法则仍然有用：如果一个命令的名称中包含 `@`，那么它不是支持的  $\text{\LaTeX}$  语言的一部分——而且它的行为有可能会在未来的版本中发生变化！如果一个命令是混合大小写的，或者在《 *$\text{\LaTeX}$ : A Document Preparation System*》中有描述，那么您可以依赖未来的  $\text{\LaTeX}$  2<sub>ε</sub> 版本支持该命令。

## 2.5 盒子命令和颜色

即使您不打算在自己的文档中使用颜色，通过注意本节中的要点，您可以确保您的类或包与 `color` 包兼容。这可能有助于那些使用您的类或包且拥有彩色打印机的人。

确保“颜色安全”的最简单方法是始终使用 L<sup>A</sup>T<sub>E</sub>X 盒子命令而不采用 T<sub>E</sub>X 原始接口，也就是说使用 `\sbox` 而弃用 `\setbox`，采用 `\mbox` 而弃用 `\hbox`，以及采用 `\parbox` 或 `minipage` 环境而不采用 `\vbox`。L<sup>A</sup>T<sub>E</sub>X 盒子命令具有的新选项，使它们现在与 T<sub>E</sub>X 原始接口命令一样强大。

举例说明可能出现问题的情况，考虑 `\ttfamily <text>` 命令，该命令下字体在“}”之前恢复，而在类似的结构 `\color{green} <text>` 中，颜色是在最终的 `}` 之后恢复的。通常，这种区别根本不重要；但考虑到诸如以下的原始 T<sub>E</sub>X 盒子定义：

```
\setbox0=\hbox{\color{green} <text>}
```

这种情况下，颜色恢复在“}”之后发生，因此它不会存储在盒子中。这可能会导致的坏影响取决于颜色的实现方式：它可以从在文档的其余部分中获得错误的颜色，到在用于打印文档的 dvi 驱动程序中引发错误。

还值得关注的是命令 `\normalcolor`。这通常只是 `\relax`（即，什么都不做），但您可以像使用 `\color` 一样使用它，将页面的某些区域（如标题或章节标题）设置为“主文档颜色”。

## 2.6 定义文本和数学字符

由于 L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub> 支持不同的编码，因此为产生符号、重音字母、复合字形等而定义的命令必须使用为这一目的所设计的命令进行定义，详见 *L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub> Font Selection*。该系统的这一部分仍在开发中，因此在进行这类任务时应格外谨慎。

此类的与编码无关的命令应使用 `\DeclareRobustCommand`。

请注意，现在不再可以在数学模式之外引用数学字体的设置：例如，`\textfont 1` 和 `\scriptfont 2` 在其他模式中不能保证被定义。

## 2.7 命令风格

新系统提供了许多命令，旨在帮助您生成结构良好、健壮且可移植的类和包文件。本节概述了如何明智且有效地使用这些命令。



### 2.7.1 加载其他文件

以下是 L<sup>A</sup>T<sub>E</sub>X 所提供，在类或包内部调用类或包命令：

```
\LoadClass          \LoadClassWithOptions  
\RequirePackage     \RequirePackageWithOptions
```

New description

原文：

1995/12/01

翻译：

2024/03/17

在此强烈建议使用这些命令替换原始的 `\input` 命令，其原因。

使用 `\input <filename>` 加载的文件不会罗列在 `\listfiles` 列表中。

利用 `\RequirePackage...` 或 `\usepackage` 命令加载宏包，即使请求多次加载，该宏包也只会加载一次。相反，如果使用 `\input` 命令引入宏包，该宏包将有可能被多次加载，其额外加载会造成时间与内存的浪费，并有可能产生奇怪的结果。

通过 `\usepackage` 或 `\RequirePackage...` 可以正常加载提供选项处理的宏包<sup>1</sup>，而使用 `\input` 加载这些宏包也可能会出现奇怪的结果。

如果模板类 `foo.sty` 使用语句 `\input baz.sty` 来引入模板类 `baz.sty`，那么用户将会收到一个警告：

```
LaTeX Warning: You have requested package 'foo',  
                but the package provides 'baz'.
```

因此，出于上述几个原因，本文不推荐使用 `\input` 加载宏包与模板类。

不幸的是，如果您正在将文件 `myclass.sty` 升级为类文件，那么您必须确保任何包含 `\input myclass.sty` 的旧文件仍然可用。

标准模板类（`article`、`book` 和 `report`）也需要满足该条件，因为许多现有的 L<sup>A</sup>T<sub>E</sub>X 2.09 文档样式包含 `\input article.sty`。该问题的解决方法是提供仅加载相应的类文件最小的文件，例如 `article.sty`、`book.sty` 和 `report.sty`。

例如，`article.sty` 只包含以下几行：

```
\NeedsTeXFormat{LaTeX2e}  
\@obsoletedefile{article.cls}{article.sty}  
\LoadClass{article}
```

---

<sup>1</sup>例如 `\usepackage[options]{package}`，译者注

您可能希望做同样的事情，或者如果您认为这样做是安全的，您可以决定只删除 `myclass.sty`。

### 2.7.2 稳健性建议

在编写包和类时，请尽可能使用 `LATEX` 命令。

因此，我们更推荐使用 `\newcommand`、`\renewcommand` 或 `\providecommand` 中的一个，而非 `\def...`。使用这样做可以减少意外重新定义命令的可能，从而产生意想不到的结果。

请使用 `\newenvironment` 或 `\renewenvironment` 定义环境，并尽量避免使用 `\def\foo{...}` 和 `\def\endfoo{...}`。

请使用 `\setlength` 来设置或更改  $\langle dimen \rangle$  或  $\langle skip \rangle$  寄存器的值。

请使用 `\sbox`、`\mbox` 和 `\parbox` 等 `LATEX` 命令来操作盒子 (box)，而尽量避免使用 `\setbox`、`\hbox` 和 `\vbox`。

请使用 `\PackageError`、`\PackageWarning` 或 `\PackageInfo` (或等效的类命令)，而尽量避免使用 `@latexerr`、`@warning` 或 `\wlog`。

对于声明选项，仍然可以通过定义 `\ds@option` 并调用 `@options` 来完成，但更推荐使用更为强大且占用内存更少的 `\DeclareOption` 和 `\ProcessOptions` 命令。所以，相比较以下命令：

```
\def\ds@draft{\overfullrule 5pt}
\@options
```

更推荐以下命令形式：

```
\DeclareOption{draft}{\setlength{\overfullrule}{5pt}}
\ProcessOptions\relax
```

这种做法可以使代码更易读，更重要的是，该代码在 `LATEX` 的升级版本中使用更不容易出现错误。

### 2.7.3 可移植性建议

编写文件时最好尽可能使文件具有良好的可移植性。为了确保这一点，其字符集需限制在标准 ASCII 编码以内；而且文件名应该最多包含八个字符（并加上三个字母的扩展名）。当然，它**绝对不能**与标准 L<sup>A</sup>T<sub>E</sub>X 发行版中的文件同名，即使其内容与对应文件相似度再高也不行。

如果本地模板类或宏包具有共同的前缀，例如诸如“Nowhere 大学”的类可能以 `unw` 开头。这有助于避免每个大学都把自己的论文模板类命名为 `thesis.cls`。

如果您依赖于 L<sup>A</sup>T<sub>E</sub>X 内核的某些特性或某个包，请指定您需要的发布日期。例如，包的错误命令是在 1994 年 6 月发布的版本中引入的，所以如果您使用该特性，则应该添加：

```
\NeedsTeXFormat{LaTeX2e}[1994/06/01]
```

### 2.7.4 监测命令的使用

为实现文档开始与结束的个性化操作，某些宏包和文档类曾经需要重定义命令 `\document` 或 `\enddocument`。而现在在“钩子命令”`\AtBeginDocument` 和 `\AtEndDocument`（详见第 4.6 节）的帮助下，已经不再需要再这么做了。再次强调，这些监测命令可以减少你的代码在将来的 L<sup>A</sup>T<sub>E</sub>X 版本中出错的可能性，并增加了你的宏包与他人的协作的可能。

不过，请注意，`\AtBeginDocument{1}` 命令中的代码是前言的一部分。因此，其在内容上有相应限制：例如其中一项非常重要的一点是，该部分代码不能进行任何排版。

New description  
原文：  
1996/12/01  
翻译：  
2024/03/18

## 3 模板类与宏包的结构

L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub> 的模板类和宏包比 L<sup>A</sup>T<sub>E</sub>X 2.09 样式文件结构条理更加规范化。其一个模板类或宏包文件的大纲如下：

**标识** 该部分文件声明自己是一个 L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub> 宏包或类，并简要描述其内容。

**初步声明** 这里文件声明了一些命令，也可以加载其他文件。通常这些命令将仅限于所声明选项中使用的代码所需的命令。

**选项** 文件声明并处理其所设置的可选参数。

**更多声明** 这是文件执行大部分工作的地方：声明新的变量、命令和字体；以及加载其他文件。

### 3.1 标识

一个类或包文件的第一步是进行自我识别。宏包文件使用以下方式进行自我识别：

```
\NeedsTeXFormat{LaTeX2e}
\ProvidesPackage{<package>}[<date> <other information>]
```

例如：

```
\NeedsTeXFormat{LaTeX2e}
\ProvidesPackage{latexsym}[1994/06/01 Standard LaTeX package]
```

模板类文件使用以下方式进行自我识别：

```
\NeedsTeXFormat{LaTeX2e}
\ProvidesClass{<class-name>}[<date> <other information>]
```

例如：

```
\NeedsTeXFormat{LaTeX2e}
\ProvidesClass{article}[1994/06/01 Standard LaTeX class]
```

⟨日期⟩应以“YYYY/MM/DD”的形式给出，并且在使用可选参数时必须存在（对于 `\NeedsTeXFormat` 命令也是如此）。任何与此语法不符的差异都将导致低级  $\text{\TeX}$  底层错误——这些命令默认所使用的均为有效语法以加快包或类的日常使用速度，但这意味着开发人员犯错的情况不在考虑范围之内！

每当用户在其 `\documentclass` 或 `\usepackage` 命令中指定日期时，都会检查此日期。例如，如果写入：

New description  
原文：  
1998/06/19  
翻译：  
2024/03/18

```
\documentclass{article}[1995/12/23]
```

那么不同地点的用户将收到一个警告说他们的 `article` 副本已过时。

当使用类时，会显示类的描述。而使用包时，描述将放入日志文件中。这些描述也会被 `\listfiles` 命令显示。除了标准 `LaTeX` 发行版之外，`Standard LaTeX` 绝对不能用于的其他任何文件的标识符中。

### 3.2 使用模板类与宏包

`LaTeX 2.09` 样式文件与 `LaTeX 2ε` 宏包和模板类之间的第一个主要区别在于 `LaTeX 2ε` 支持 模块化 组建模式，即从小的构建块构建文件，而不是使用大的单个文件作为其搭建方式。

一个 `LaTeX` 宏包或模板类可以加载一个宏包，如下所示：

```
\RequirePackage[<options>]{<package>}[<date>]
```

例如：

```
\RequirePackage{ifthen}[1994/06/01]
```

此命令具有与作者命令 `\usepackage` 相同的语法。它允许包或类使用其他包提供的功能。例如，通过加载 `ifthen` 包，包作者可以使用该包提供的“`if...then...else...`”命令。

一个 `LaTeX` 模板类可以加载另一个模板类，如下所示：

```
\LoadClass[<options>]{<class-name>}[<date>]
```

例如：

```
\LoadClass[twocolumn]{article}
```

该命令具有与作者命令 `\documentclass` 相同的语法。它允许类基于另一个类的语法和外观。例如，通过加载 `article` 类，类作者只需更改他们不喜欢的 `article` 的部分，而不是从头开始编写一个新类。

在常见情况下，您可以使用以下命令简单地加载一个类或包文件，使其具有与当前类使用的完全相同的选项：

New feature  
原文：  
1995/12/01  
译文：  
2024/03/18

```
\LoadClassWithOptions{<class-name>}[<date>]  
\RequirePackageWithOptions{<package>}[<date>]
```

例如：

```
\LoadClassWithOptions{article}  
\RequirePackageWithOptions{graphics}[1995/12/01]
```

### 3.3 选项参数声明

L<sup>A</sup>T<sub>E</sub>X 2.09 样式与 L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub> 宏包与模板类之间的另一个主要区别在于选项参数处理。当前，宏包与模板类均可以声明选项参数，而作者在使用这些宏包与模板类是可以设定这些参数的值；例如，`article` 模板类中声明了 `twocolumn` 参数选项。注意，选项参数名称所使用字符应限制在“L<sup>A</sup>T<sub>E</sub>X 名称”中允许的字符范围内；尤其是不能包含任何控制序列。

New description

原文：

1998/12/01

翻译：

2024/03/23

选项参数的声明如下：

```
\DeclareOption{<option>}{<code>}
```

例如，`graphics` 包的选项参数 `dvips` 的实现如下（该选项有略微简化）：

```
\DeclareOption{dvips}{\input{dvips.def}}
```

这表明 `\usepackage[dvips]{graphics}` 命令会将文件 `dvips.def` 加载到文件中。另一个例子是 `article` 模板类中声明了 `a4paper` 选项参数，该参数的作用是指定 `\paperheight` 和 `\paperwidth` 的值来规范版面设置，其实现如下：

```
\DeclareOption{a4paper}{%  
\setlength{\paperheight}{297mm}%  
\setlength{\paperwidth}{210mm}%  
}
```

用户有时可能会指定一个模板类或宏包未明确声明的选项参数。对此，在默认情况下，模板类会报一个警告而宏包则会报一个错误；若不希望如此，可以按如下方式修改其报错原则：

```
\DeclareOption*{\code}
```

例如，编写宏包 `fred` 时加入以下语句：

```
\DeclareOption*{%  
\PackageWarning{fred}{Unknown option '\CurrentOption'}%  
}
```

可以达到对未知选项参数报警告而非报错。例如在未指定选项参数 `foo` 的情况下，使用命令 `\usepackage[foo]{fred}` 将会报警告 `Package fred Warning: Unknown option 'foo'`。而一个实际的例子是宏包 `fontenc` 在设定参数选项 `\ENC` 时加载 `\ENC`enc.def 的命令，该命令如下：

```
\DeclareOption*{%  
\input{\CurrentOption enc.def}%  
}
```

使用 `\PassOptionsToPackage` 或 `\PassOptionsToClass` 命令可以将选项传递给另一个包或类（请注意，这是一项专门的操作，仅适用于选项名称）。例如以下命令将所有未知选项都传递给 `article` 类：

```
\DeclareOption*{%  
\PassOptionsToClass{\CurrentOption}{article}%  
}
```

请确保在该条命令之后加载该类，否则这些选项参数将被无视！

以上是对选项参数声明的描述，而处理执行则在下文提及。处理文件调用时的选项参数应该使用以下命令：

```
\ProcessOptions\relax
```

该命令将已经完成声明并在调用时指定的每个选项参数的 `\code`（有关执行方式的详细信息请参见第 4.7 节）。

举个例子，在 `jane` 宏包文件包含以下命令：

New description  
原文：  
1998/12/01  
翻译：  
2024/03/24

```

\DeclareOption{foo}{\typeout{Saw foo.}}
\DeclareOption{baz}{\typeout{Saw baz.}}
\DeclareOption*{\typeout{What's \CurrentOption?}}
\ProcessOptions\relax

```

当使用 `\usepackage[foo,bar]{jane}` 引入该宏包时，使用者会收到 “Saw foo.” 和 “What’s bar?” 这样两条消息。

### 3.4 最小模板类

模板类与宏包的大部分工作是定义新命令或改变文档的外观。这些工作往往在宏包的主体中达成，这涉及到诸如 `\newcommand`、`\setlength` 之类的命令。

LaTeX 2<sub>ε</sub> 提供了几个新命令来帮助类和包的编写者；这些命令将在第 4 节中详细描述。

每个模板类文件都必须包含四个要素：

- `\normalsize` 的定义
- `\textwidth` 的值
- `\textheight` 的值
- 页码规范

因此，一个最小文档类文件<sup>2</sup>形式如下：

```

\NeedsTeXFormat{LaTeX2e}
\ProvidesClass{minimal}[1995/10/30 Standard LaTeX minimal class]
\renewcommand{\normalsize}{\fontsize{10pt}{12pt}\selectfont}
\setlength{\textwidth}{6.5in}
\setlength{\textheight}{8in}
\pagenumbering{arabic}      % 即使这个模板类不显示页码
                           % 也需要相应的页码规范

```

然而，这个模板类不支持脚注、页边注、浮动体等，也不提供诸如 `\rm` 这样的双字母字体命令；因此，大多数类会提供更多的内容！

---

<sup>2</sup>该类现已包含在标准发行版中，即 `minimal.cls`。



### 3.5 示例：本地信件模板类

一个公司可能会有自己的信函类，用于编写该公司风格的信函。本节展示了这样一个类的简单实现，尽管一个真实的类会需要更多的结构。

首先声明该模板类为 `neplet.cls`。

```
\NeedsTeXFormat{LaTeX2e}
\ProvidesClass{neplet}[1995/04/01 NonExistent Press letter class]
```

接下来，这部分将所有选项传递给 `letter` 模板类，并使用使用 `a4paper` 选项参数加载 `letter` 模板类。

```
\DeclareOption*{\PassOptionsToClass{\CurrentOption}{letter}}
\ProcessOptions\relax
\LoadClass[a4paper]{letter}
```

为了使用公司信头，该模板类重定义 `firstpage` 页面样式：这是信函第一页所使用的页面样式。

```
\renewcommand{\ps@firstpage}{%
\renewcommand{\@oddhead}{\langle letterhead goes here \rangle}%
\renewcommand{\@oddfoot}{\langle letterfoot goes here \rangle}%
}
```

这样就完成了该信件模板类的编写！

### 3.6 示例：新闻简报模板类

使用  $\text{\LaTeX}$ ，利用 `article` 模板类变体，可以完成简单的新闻简报排版。首先声明该模板类为 `smplnews.cls`。

```
\NeedsTeXFormat{LaTeX2e}
\ProvidesClass{smplnews}[1995/04/01 The Simple News newsletter class]

\newcommand{\headlinecolor}{\normalcolor}
```

以下命令将大多数指定的选项参数传递给 `article` 类：除了选项参数 `onecolumn` 和 `green`，其中 `onecolumn` 选项参数将不被使用，而 `green` 选项参数会将标题设为绿色。

```
\DeclareOption{onecolumn}{\OptionNotUsed}
\DeclareOption{green}{\renewcommand{\headlinecolor}{\color{green}}}

\DeclareOption*{\PassOptionsToClass{\CurrentOption}{article}}

\ProcessOptions\relax
```

然后使用选项参数 `twocolumn` 加载类 `article`。

```
\LoadClass[twocolumn]{article}
```

新闻简报需要加载 `color` 宏包服务于彩色打印。设备驱动程序选项参数不由该模板类指定，而应该由使用 `smplnews` 模板类的用户指定。

```
\RequirePackage{color}
```

随后该模板类重定义 `\maketitle` 命令，以 72pt Helvetica 粗斜体格式<sup>3</sup> 的适当颜色字体产生标题。

```
\renewcommand{\maketitle}{%
\twocolumn[%
\fontsize{72}{80}\fontfamily{phv}\fontseries{b}%
\fontshape{sl}\selectfont\headlinecolor
\@title
]%
}
```

重定义 `\section` 命令并关闭节编号。

```
\renewcommand{\section}{%
\@startsection
{section}{1}{0pt}{-1.5ex plus -1ex minus -.2ex}%
```

---

<sup>3</sup>原文：72pt Helvetica bold oblique，译者注

```
{1ex plus .2ex}{\large\sffamily\slshape\headlinecolor}%
}
```

```
\setcounter{secnumdepth}{0}
```

设置了其他三个重要参数。

```
\renewcommand{\normalsize}{\fontsize{9}{10}\selectfont}
\setlength{\textwidth}{17.5cm}
\setlength{\textheight}{25cm}
```

实际使用中，一个模板类会比这个更复杂：它提供设置期刊号、文章作者、页面样式等参数的命令。这个框架只是一个初学者样例。实际上，`ltnews` 模板类文件不比这个复杂多少。

## 4 书写模板类与宏包的命令

这一章节将简言概括写模板类与宏包的每个新命令。若需要了解该新系统的其他方面，请参考《*L<sup>A</sup>T<sub>E</sub>X: A Document Preparation System*》、《*The L<sup>A</sup>T<sub>E</sub>X Companion*》以及《*L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub> for Authors*》。

### 4.1 标记识别

首先需要讨论用于识别所编写模板类或宏包文件的命令。

```
\NeedsTeXFormat{<format-name>}[<release-date>]
```

该命令告诉 T<sub>E</sub>X 使用名为 `<format-name>` 的格式处理此文件，同时可选参数 `<release-date>` 进一步指定所需格式的最早发布日期。当格式的发布日期早于指定的日期时，将生成警告。标准 `<format-name>` 是 `LaTeX2e`。如果有需要的话，`<release-date>` 所指定的日期必须以 `YYYY/MM/DD` 的形式给出。

例如：

```
\NeedsTeXFormat{LaTeX2e}[1994/06/01]
```

```
\ProvidesClass {<class-name>} [ <release-info>]  
\ProvidesPackage {<package-name>} [ <release-info>]
```

两条指令声明了当前文件包含文档类  $\langle class-name \rangle$  或宏包  $\langle package-name \rangle$  的定义。

$\langle release-info \rangle$  可以不使用，若使用  $\langle release-info \rangle$  则要包含以下信息：

- 以 YYYY/MM/DD 的形式所表示的  $\langle format-name \rangle$  版本发布日期；
- （可有可无）日期后用一个空格隔开的简短描述，该描述中可能包括版本号。

上述语法须严格遵循，以便该文件被 `\LoadClass` 或 `\documentclass`（对于模板类）或 `\RequirePackage` 或 `\usepackage`（对于宏包）调用时检查发布版本是否兼容<sup>4</sup>。

整个  $\langle release-info \rangle$  信息都会被 `\listfiles` 显示，因此不应设置太长。

示例：

```
\ProvidesClass{article}[1994/06/01 v1.0 Standard LaTeX class]  
\ProvidesPackage{ifthen}[1994/06/01 v1.0 Standard LaTeX package]
```

```
\ProvidesFile {<file-name>} [ <release-info>]
```

这与前两个命令类似，但此处必须给出完整的文件名，包括扩展名。它用于声明除主类和包文件之外的任何文件。

示例：

```
\ProvidesFile{Tlenc.def}[1994/06/01 v1.0 Standard LaTeX file]
```

请注意，短语 **Standard LaTeX** 绝不能在标准 L<sup>A</sup>T<sub>E</sub>X 发行版之外的任何文件的标识符中使用。

## 4.2 加载文件

这组命令可用于基于现有的类或包构建自己的文档模板类或宏包。

<sup>4</sup>原文：to test that the release is not too old，译者注

New feature

添加日期：

1995/12/01

翻译：

2024/03/26

`\RequirePackage [<options-list>] {<package-name>} [<release-info>]`  
`\RequirePackageWithOptions {<package-name>} [<release-info>]`

宏包和模板类应该使用这些命令来加载其他包。

`\RequirePackage` 的使用方式与作者命令 `\usepackage` 相同。

例如：

```
\RequirePackage{ifthen}[1994/06/01]
\RequirePackageWithOptions{graphics}[1995/12/01]
```

`\LoadClass [<options-list>] {<class-name>} [<release-info>]`  
`\LoadClassWithOptions {<class-name>} [<release-info>]`

这些命令仅用于类文件中，不能在宏包文件中使用，且在类文件中能且只能使用一次。

New feature  
1995/12/01

`\LoadClass` 的使用方式与使用 `\documentclass` 加载类文件相同。

例如：

```
\LoadClass{article}[1994/06/01]
\LoadClassWithOptions{article}[1995/12/01]
```

这两个带有 `WithOptions` 后缀的版本只是以当前文件（类或包）使用的选项加载模板类（或宏包）文件。有关其使用的进一步讨论，请参见下文 4.5。

New feature  
添加日期：  
1995/12/01  
翻译：  
2024/03/26

### 4.3 选项参数声明

以下命令处理文档类和包的选项的声明和处理。每个选项名称必须是一个“`LaTeX`名称”。

New description  
原文：  
1998/12/01  
翻译：  
2024/03/26

有一些命令专门设计用于这些命令的 `<code>` 参数中（见下文）。

`\DeclareOption {<option-name>} {<code>}`

模板类或红包中使用该命令可以完成声明选项参数 `<option-name>`。

其中  $\langle code \rangle$  参数包含了当该选项为类或包指定时要执行的代码；它可以包含任何有效的 L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub> 构造。

示例：

```
\DeclareOption{twoside}{\@twoside>true}
```

`\DeclareOption* {\langle code \rangle}`

以上代码中  $\langle code \rangle$  阐明了若使用模板类或宏包未声明的选项参数所要执行的代码，此代码称为“默认选项代码”，它可以包含任何有效的 L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub> 构造。

如果一个模板类文件不包含 `\DeclareOption*`，则在默认情况下，若使用该模板类未声明的选项参数，该参数将会被静默传递给所有包（与该类的已指定和声明的选项一样）。

如果一个宏包文件不包含 `\DeclareOption*`，则默认情况下，使用该宏包未声明的选项参数会报错。

## 4.4 选项参数代码内的命令

以下两条命令只能在 `\DeclareOption` 或 `\DeclareOption*` 的  $\langle code \rangle$  参数中使用。其他常用于  $\langle code \rangle$  的命令可以在接下来的几个子节中找到。

`\CurrentOption`

该命令展开为当前选项的名称。

`\OptionNotUsed`

该命令会将当前选项参数被添加到“未使用选项参数”列表中。

现在可以在这些  $\langle code \rangle$  参数中包含井号 (#)，而无需进行特殊处理（以前需要使用双井号）。

New feature  
添加日期：  
1995/06/01  
翻译：  
2024/03/26

## 4.5 选项参数传递

这是两条在 `\DeclareOption` 或 `\DeclareOption*` 的 `<code>` 参数中也非常有用的命令：

```
\PassOptionsToPackage {<options-list>} {<package-name>}  
\PassOptionsToClass {<options-list>} {<class-name>}
```

命令 `\PassOptionsToPackage` 将 `<options-list>` 中的选项参数名称传递给包 `<package-name>`。这意味着它将 `<option-list>` 添加到未来任何 `\RequirePackage` 或 `\usepackage` 命令中用于包 `<package-name>` 的选项参数列表中。

例如：

```
\PassOptionsToPackage{foo,bar}{fred}  
\RequirePackage[baz]{fred}
```

该命令等效于：

```
\RequirePackage[foo,bar,baz]{fred}
```

类似地，`\PassOptionsToClass` 可以在模板类文件中使用，该命令将选项参数传递给使用 `\LoadClass` 加载的模板类。

以上这两条命令的效果和使用应该与对应的两条加载命令配合使用（在前文 4.2 部分已提及）：

```
\LoadClassWithOptions  
\RequirePackageWithOptions
```

命令 `\RequirePackageWithOptions` 的行为与 `\RequirePackage` 类似，但包含 `WithOptions` 的命令始终以与当前模板类或宏包使用的相同选项列表加载所需的宏包，而不使用 `\PassOptionsToPackage` 明确提供或传递的任何选项。

设计 `\LoadClassWithOptions` 命令的主要目的是允许简单地在一个基础模板类上搭建一个自定义的类，例如：

```
\LoadClassWithOptions{article}
```

New description  
原文：  
1995/12/01  
翻译：  
2024/03/27

与利用其他命令的等效操作作比较：

```
\DeclareOption*{\PassOptionsToClass{\CurrentOption}{article}}
\ProcessOptions\relax
\LoadClass{article}
```

在上述用法中，效果或多或少相同<sup>5</sup>，但前者输入更少；而且 `\LoadClassWithOptions` 方法运行速度略快。

然而，如果类声明了自己的选项，则这两种构造是不同的。例如，比较以下自定义 `landscape` 参数选项两个例子：

使用 `\LoadClassWithOptions`：

```
\DeclareOption{landscape}{\@landscapetrue}
\ProcessOptions\relax
\LoadClassWithOptions{article}
```

使用 `\LoadClassWith`：

```
\DeclareOption{landscape}{\@landscapetrue}
\DeclareOption*{\PassOptionsToClass{\CurrentOption}{article}}
\ProcessOptions\relax
\LoadClass{article}
```

在第一个示例中，当前模板类使用 `landscape` 选项参数时，`article` 模板类将以 `landscape` 选项参数加载。相比之下，在第二个示例中，`article` 类永远不会以 `landscape` 选项参数加载，因为在这种情况下，`article` 仅通过默认选项处理程序传递选项，但是对于 `landscape`，这个处理程序不起作用，因为该选项已被显式声明。

## 4.6 延迟代码

这两个命令也主要设计用于 `\DeclareOption` 或 `\DeclareOption*` 的 `<code>` 参数中。

---

<sup>5</sup>原文：more or less the same, 译者注



```
\AtEndOfClass {\code}  
\AtEndOfPackage {\code}
```

这些命令声明的  $\langle code \rangle$  会在整个当前模板类或宏包文件处理完毕后保存，并在之后执行。

这些命令可重复使用：参数中的代码按声明顺序存储（并在稍后执行）。

```
\AtBeginDocument {\code}  
\AtEndDocument {\code}
```

这些命令声明的  $\langle code \rangle$  会在 L<sup>A</sup>T<sub>E</sub>X 执行 `\begin{document}` 与 `\end{document}` 时保存并执行。

`\AtBeginDocument` 的参数中指定的  $\langle code \rangle$  在 `\begin{document}` 代码的末尾执行，在字体选择表设置完成后执行。因此，在所有关于排版的准备工作都做好，并且当前字体设置为一般字体后，推荐将其他需要执行的代码放到这个部分中来。

`\AtBeginDocument` 监测不应当用于执行任何排版命令，因为排版结果会变得不可预测。

`\AtEndDocument` 的参数中指定的  $\langle code \rangle$  在 `\end{document}` 代码开始执行时执行，在最终页面完成之前以及处理任何未完成的浮动环境之前执行。如果其中一些  $\langle code \rangle$  需要在这两个过程之后执行，应在  $\langle code \rangle$  的适当位置包含 `\clearpage`。

可以重复使用这些命令：参数中的代码按使用的先后顺序存储（并稍后执行）。

```
\AtBeginDvi {\specials}
```

这些命令将对应内容保存在一个盒子寄存器中，这些内容会写入 `.dvi` 文件<sup>6</sup> 的开头，以用于生成文件首页。

该命令不应用于向 `.dvi` 文件中添加任何排版材料。

该命令可重复使用。

---

<sup>6</sup>DVI文件是TeX系统生成的设备无关文件（Device Independent File），它包含了文档的排版信息，但不包含与具体输出设备相关的信息，译者注，来自于ChatGPT

New description  
原文：  
1995/12/01  
翻译：  
2024/03/27

New feature  
1994/12/01

## 4.7 选项参数处理

### `\ProcessOptions`

该命令会执行所有选项参数对应的 `<code>`。

这里将首先介绍 `\ProcessOptions` 在宏包中的执行方式，并随后介绍其在模板类文件中执行方式的差异。

理解 `\ProcessOptions` 在宏包中的工作细节，需要首先了解 局域选项参数 与 全局选项参数 的差异：

- **局部选项参数** 是在任何以下命令的 `<options>` 参数中明确指定的针对此特定包的选项参数，其指定方式有：

```
\PassOptionsToPackage{<options>} \usepackage[<options>]  
\RequirePackage[<options>]
```

- **全局选项参数** 是作者在 `\documentclass[<options>]` 的 `<options>` 参数中指定的其他选项。

例如，以下声明了一个 `article` 文档并无参加载了 `gerhardt` 宏包：

```
\documentclass[german,twocolumn]{article}  
\usepackage{gerhardt}
```

而 `gerhardt` 宏包以以下参数选项调用 `fred` 宏包：

```
\PassOptionsToPackage{german,dvips,a4paper}{fred}  
\RequirePackage[errorshow]{fred}
```

此时则有

- `fred` 接收的局域选项参数是 `german`、`dvips`、`a4paper` 和 `errorshow`；
- `fred` 接收的全局选项参数只有 `twocolumn`。

调用 `\ProcessOptions` 时，会发生如下情况：

- 首先，对于在 `fred.sty` 中由 `\DeclareOption` 声明的每个选项选项参数，它会查看该选项是否是 `fred` 的全局选项参数或局部选项参数：如果是，则执行相应的代码。

代码执行遵循对应选项参数在 `fred.sty` 中声明顺序。

- 然后，对于其他 局部 选项，若命令 `\ds@{option}` 已经在其他地方定义（而非由 `\DeclareOption` 定义），则执行该命令；否则，执行“默认选项参数代码”。若没有声明默认选项参数代码则报错。

这是按照这些选项参数被指定的顺序进行的。

在整个过程中，系统确保选项参数声明的代码最多只执行一次。

回到示例，假设 `fred.sty` 包含以下内容：

```
\DeclareOption{dvips}{\typeout{DVIPS}}
\DeclareOption{german}{\typeout{GERMAN}}
\DeclareOption{french}{\typeout{FRENCH}}
\DeclareOption*{\PackageWarning{fred}{Unknown '\CurrentOption'}}
\ProcessOptions\relax
```

则该文档处理的结果如下：

```
DVIPS
GERMAN
Package fred Warning: Unknown 'a4paper'.
Package fred Warning: Unknown 'errorshow'.
```

请注意以下内容：

- `fred.sty` 中选项参数 `dvips` 在 `german` 之前声明，所以执行时也是先执行 `dvips` 后执行 `german`；
- `german` 的代码只执行一次，即仅在处理声明选项参数时执行；
- 根据 `\DeclareOption*` 声明的代码，`a4paper` 和 `errorshow` 选项参数报按其调用顺序报警告，而 `twocolumn` 作为全局选项参数则没有报警告。

在模板类文件中，`\ProcessOptions` 的工作方式相同，只是模板类将 所有选项参数都视作局部选项参数；而 `\DeclareOption*` 的默认值是 `\OptionNotUsed` 而不是错误。

请注意，下文将介绍 `\ProcessOptions` 的一个 “\*-形式”，故在此建议在之前使用非星号形式后加上 `\relax`，如前面的示例中所示。这样可以防止不必要的上下文冲突以及可能会产生误导错误消息。

New description

原文:

1995/12/01

翻译:

2024/03/27

`\ProcessOptions*`  
`\@options`

该命令与 `\ProcessOptions` 功能类似，但顺序上会按照调用命令中指定的顺序执行选项参数，而非模板类或宏包中声明的顺序。在宏包中该命令会优先处理全局选项参数。

为了简化将旧的文档样式更新为  $\text{\LaTeX 2}_\epsilon$  类文件的任务， $\text{\LaTeX 2.09}$  中的 `@options` 命令已被改为等效于此。<sup>7</sup>

`\ExecuteOptions {<options-list>}`

该命令仅仅为 `<options-list>` 中的每个选项参数依次执行命令 `\ds@<option>`（若此命令未定义，则该选项将被静默忽略）。

该命令可用于在 `\ProcessOptions` 之前提供“默认选项列表”。例如，假设在一个类文件中，您希望设置默认设计为：双面打印；11pt 字体；双栏。那么可以用该命令指定：

```
\ExecuteOptions{11pt,twoside,twocolumn}
```

## 4.8 安全的文件操作

这些命令用于处理文件输入，以确保当所请求文件不存在时，处理更为人性化。

`\IfFileExists {<file-name>} {<true>} {<false>}`

若所指定文件 `file-name` 存在则执行 `<true>`，若所指定文件 `file-name` 不

<sup>7</sup>根据第 2 章内容，应避免在新项目中使用旧版命令，译者注

存在则执行  $\langle false \rangle$ 。

该命令 不会 导入对应文件。

`\InputIfFileExists { $\langle file-name \rangle$ } { $\langle true \rangle$ } { $\langle false \rangle$ }`

若所指定文件 `file-name` 存在，该命令将先执行  $\langle true \rangle$  随后当即导入该文件。反之，若不存在则执行  $\langle false \rangle$ 。

该命令内部实现使用 `\IfFileExists`。

## 4.9 报错与其他代码

这些命令用于第三方模板类和宏包向使用者报告错误或提供信息。

`\ClassError { $\langle class-name \rangle$ } { $\langle error-text \rangle$ } { $\langle help-text \rangle$ }  
\PackageError { $\langle package-name \rangle$ } { $\langle error-text \rangle$ } { $\langle help-text \rangle$ }`

这些是报错命令。报错时会显示  $\langle error-text \rangle$  的内容，并显示 `?` 作为错误提示符。如果用户键入 `h`，则会显示  $\langle help-text \rangle$ 。

在  $\langle error-text \rangle$  和  $\langle help-text \rangle$  中，可以使用 `\protect` 来阻止命令展开<sup>8</sup>，`\MessageBreak` 可以实现打印信息换行，而 `\space` 则打印一个空格。

请注意， $\langle error-text \rangle$  会自动在句尾添加一个句号，因此不要将其放入参数中。

例如：

```
\newcommand{\foo}{F00}  
\PackageError{ethel}{%  
Your hovercraft is full of eels,\MessageBreak  
and \protect\foo\space is \foo  
}%  
Oh dear! Something's gone wrong.\MessageBreak  
\space \space Try typing \space <return>  
\space to proceed, ignoring \protect\foo.  
}
```

---

<sup>8</sup>命令展开指的是利用宏替换将命令展开为层次更低的形式，译者注

将会给出这样一个报错：

```
! Package ethel Error: Your hovercraft is full of eels,  
(ethel)                and \foo is F00.
```

See the ethel package documentation for explanation.

若用户键入 `h`，则会弹出这样的帮助信息：

```
Oh dear! Something's gone wrong.  
Try typing <return> to proceed, ignoring \foo.
```

```
\ClassWarning {\langle class-name \rangle} {\langle warning-text \rangle}  
\PackageWarning {\langle package-name \rangle} {\langle warning-text \rangle}  
\ClassWarningNoLine {\langle class-name \rangle} {\langle warning-text \rangle}  
\PackageWarningNoLine {\langle package-name \rangle} {\langle warning-text \rangle}  
\ClassInfo {\langle class-name \rangle} {\langle info-text \rangle}  
\PackageInfo {\langle package-name \rangle} {\langle info-text \rangle}
```

这四个 `Warning` 命令与错误命令类似，不同之处在于它们仅在屏幕上报警告，没有错误提示。

前两个 `Warning` 版本还显示造成警告的行号，而后两个 `WarningNoLine` 版本则不显示行号。

这两个 `Info` 命令类似，它们仅起到在传递文件中记录信息的作用，包括行号。这两个命令没有 `NoLine` 版本。

在 `\langle warning-text \rangle` 和 `\langle info-text \rangle` 中，同样可以使用 `\protect` 来阻止命令展开，利用 `\MessageBreak` 来换行，并利用 `\space` 打印一个空格。另外，这些文本也不应以句号结尾，因为句号会自动添加。

## 4.10 定义命令

L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub> 提供了几种新方法用于定义与重定义命令，以用于模板类与宏包文件。

这些命令的“`*`-形式”应当用于短命令，相对于 T<sub>E</sub>X 标准来说。这有助于在不包含整段文字的参数中定位错误。

New feature  
添加日期:  
1994/12/01  
翻译:  
2024/03/27

```
\DeclareRobustCommand {\langle cmd\rangle} [\langle num\rangle] [\langle default\rangle] {\langle definition\rangle}
\DeclareRobustCommand* {\langle cmd\rangle} [\langle num\rangle] [\langle default\rangle] {\langle definition\rangle}
```

该命令与 `\newcommand` 接受相同的参数，但它声明了一个稳健命令，即使  $\langle definition \rangle$  中的某些代码是脆弱的。您可以使用此命令定义新的稳健命令，或重定义现有命令并增强其容错率。如果重新定义了命令，则会在传输文件中记录日志。

例如，自定义命令 `\seq` 定义如下：

```
\DeclareRobustCommand{\seq}[2][n]{%
  \ifmmode
    #1_{1}\ldots#1_{#2}%
  \else
    \PackageWarning{fred}{You can't use \protect\seq\space in text}%
  \fi
}
```

根据以上定义，即使 `\ifmmode` 不能在移动参数<sup>9</sup>中使用，命令 `\seq` 也可以，例如：

```
\section{Stuff about sequences $\seq{x}$}
```

另请注意，在定义的开头 `\ifmmode` 前面没有必要放置 `\relax`；这是因为这个 `\relax` 在错误的时候提供了内部扩展的保护。

```
\CheckCommand {\langle cmd\rangle} [\langle num\rangle] [\langle default\rangle] {\langle definition\rangle}
\CheckCommand* {\langle cmd\rangle} [\langle num\rangle] [\langle default\rangle] {\langle definition\rangle}
```

这两条命令与 `\newcommand` 接受相同的参数，但是与定义  $\langle cmd \rangle$  不同，它只是检查当前定义的  $\langle cmd \rangle$  是否完全与  $\langle definition \rangle$  给出的相同。如果这些定义不同，则会引发错误。

这个命令对于在包开始改变命令的定义之前检查系统状态很有用。它允许您检查特别是没有其他包重新定义相同命令。

<sup>9</sup>移动参数指嵌套在格式里面的格式，例如 `\section{开平方 $\sqrt{x}$ 的应用}`，译者注，源自ChatGPT

## 4.11 传递参数

在处理（即移动）移动参数时设置保护的功能已重新实现，从 `.aux` 文件将信息写入其他文件（如 `.toc` 文件）的方法也已重新实现。具体细节参见文件 `ltdefns.dtx`。

我们希望这些变更不会影响太多的宏包。

New description

原文:

1994/12/01

翻译:

2024/03/27

## 5 杂项命令以及其他

### 5.1 版面参数

`\paperheight`  
`\paperwidth`

这两个参数通常由文档类设置为所使用的纸张大小。这应该是实际的纸张尺寸，不像 `\textwidth` 和 `\textheight` 是在页边距内主文本正文的尺寸。（译者案例：利用 `\the\paperheight` 与 `\the\paperwidth` 获取本文的参数为 794.96999pt 与 614.295pt，其中  $1\text{pt} = 1/72 \text{ inch} \sim 0.35146\text{mm}$ ）

### 5.2 大小写转换

`\MakeUppercase {text}`  
`\MakeLowercase {text}`

$\text{T}_{\text{E}}\text{X}$  提供了两个原始命令 `\uppercase` 和 `\lowercase` 用于改变文本的大小写。有时在文档类中会使用这些命令，例如在页眉中将信息全部转换为大写。

不幸的是，这些  $\text{T}_{\text{E}}\text{X}$  原始命令不会改变通过命令 `\ae` 或 `\aa` 访问的字符的大小写。为了解决这个问题， $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$  提供了两个新命令 `\MakeUppercase` 和 `\MakeLowercase` 来实现。

例如：

New feature

添加日期:

1995/06/01

翻译:

2024/03/27



|              |            |
|--------------|------------|
| —ABCDæÅßCE—  | ABCDæÅßCE  |
| —abcdæÅßCE—  | abcdæÅßCE  |
| —ABCDÆÅSSCE— | ABCDÆÅSSCE |
| —abcdæåßœ—   | abcdæåßœ   |

命令 `\MakeUppercase` 和 `\MakeLowercase` 本身是健壮的，但它们具有移动参数。而这些命令使用了  $\mathrm{T}_{\mathrm{E}}\mathrm{X}$  原始命令 `\uppercase` 和 `\lowercase`，因此具有一些意想不到的“特性”。特别地，它们会改变文本参数中的所有内容（除了控制序列名称中的字符）：这包括数学公式、环境名称和标签名称。

例如：

```
\MakeUppercase{$x+y$ in \ref{foo}}
```

会将  $x + y$  转化为  $X + Y$  并且报警告：

```
LaTeX Warning: Reference ‘F00’ on page ... undefined on ...
```

从长远来看，我们希望使用全大写字体而不是像 `\MakeUppercase` 这样的命令，但目前不可能，因为这样的字体不存在。

为使大小写转换工作得相当好，并且为了提供正确的连字符， $\mathrm{L}^{\mathrm{A}}\mathrm{T}_{\mathrm{E}}\mathrm{X}\ 2_{\epsilon}$  在整个文档中必须使用相同的固定大小写转换表。所使用的表是为字体编码  $\mathrm{T}_{\mathrm{1}}$  设计的；这在所有拉丁字母的标准  $\mathrm{T}_{\mathrm{E}}\mathrm{X}$  字体中效果很好，但在使用其他字母表时会导致问题。

New description

原文:

1995/12/01

翻译:

2024/03/27

### 5.3 “book” 模板类中的 “openany” 选项参数

`openany` 选项参数允许章节等标题在左侧页面上出现。之前，该选项仅对 `\chapter` 和 `\backmatter` 起作用。而现在添加了其对 `\part`、`\frontmatter` 和 `\mainmatter` 的作用。

New description

原文:

1996/06/01

翻译:

2024/03/27

### 5.4 更好的用户自定义数学显示环境

\ignorespacesafterend

如下方法简洁地定义了一个将文本显示为编号方程的环境：

New feature

添加日期:

1996/12/01

New description

原文:

2003/12/01

翻译:

2024/03/27

```

\newenvironment{texreqn}
{\begin{equation}
\begin{minipage}{0.9\linewidth}}
{\end{minipage}
\end{equation}}

```

然而，如果您尝试过这样做，那么您可能已经注意到，当在段落中间使用时，它不会完美地工作，因为在环境后的第一行开头会出现一个单词间距。

现在有一个额外的命令（名称非常长），您可以使用它来避免这个问题；它应该插入如下所示：

```

\newenvironment{texreqn}
{\begin{equation}
\begin{minipage}{0.9\linewidth}}
{\end{minipage}
\end{equation}
\ignorespacesafterend}

```

该命令同时还有其他用途。

## 5.5 标准化间距

\normalsfcodes

应该使用该命令来恢复影响单词、句子等间距的参数的正常设置。

此功能的一个重要用途是纠正 Donald Arseneau 报告的一个问题，即每当页面分页时，如果正在生效局部设置空格代码，则页眉中的标点可能存在问题（在所有已知的  $\text{T}_{\text{E}}\text{X}$  格式中都是如此）。这些空格代码会被例如命令 `\frenchspacing` 和 `verbatim` 环境所改变。

通常它会在 `\begin{document}` 中自动得到正确的定义，因此不需要显式设置；然而，如果在类文件中显式地将其设为非空，则默认设置将被覆盖。

New feature

添加日期:

1997/06/01

翻译:

2024/03/27

## 6 升级 L<sup>A</sup>T<sub>E</sub>X 2.09 模板类与宏包

本节描述了将现有的 L<sup>A</sup>T<sub>E</sub>X 样式升级为宏包或模板类时可能需要进行的更改，但我们将从乐观的角度开始。

许多现有的样式文件在不对文件本身进行任何修改的情况下就可以在 L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub> 中运行。当一切正常运行时，请在新创建的宏包或类文件中记录下“该文件与新的标准 L<sup>A</sup>T<sub>E</sub>X 兼容”；然后将其分发给您的用户。

### 6.1 请先做测试！

您应该做的第一件事是在“兼容模式”中测试您的样式。为了做到这一点，您唯一需要做的更改可能是将文件的扩展名更改为 `.cls`：只有在您的文件被用作主文档样式时才应该进行此更改。现在，在没有进行其他修改的情况下，在使用您的文件的文档上运行 L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub>。这假定您有一个适当的文件集合，用于测试您的样式文件提供的所有功能。（如果没有，现在是制作一个的时候了！）

您现在需要更改测试文档文件，使其成为 L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub> 文档：有关如何执行此操作的详细信息，请参阅 *L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub> for Authors*，然后再次尝试它们。现在您已经在 L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub> 原生模式和 L<sup>A</sup>T<sub>E</sub>X 2.09 兼容模式下尝试了测试文档。

### 6.2 故障排除

If your file does not work with L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub>, there are two likely reasons. 文件与 L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub> 不兼容有两个原因。

- L<sup>A</sup>T<sub>E</sub>X 现在具有健壮的、定义良好的设计者接口用于选择字体，这与 L<sup>A</sup>T<sub>E</sub>X 2.09 的内部机制非常不同。
- 您的样式文件可能使用了一些已更改或已移除的 L<sup>A</sup>T<sub>E</sub>X 2.09 内部命令。

When you are debugging your file, you will probably need more information than is normally displayed by L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub>. This is achieved by resetting the

counter —`errorcontextlines`— from its default value of `-1` to a much higher value, e.g. `999`.

在调试文件时，您可能需要比  $\text{\LaTeX}$  2<sub>ε</sub> 正常显示的信息更多。这可以通过将计数器 `errorcontextlines` 从默认值 `-1` 重置为更高的值来实现，例如 `999`。

### 6.3 适应兼容模式

有时，现有的一系列  $\text{\LaTeX}$  2.09 文档使得完全放弃旧命令变得不方便或不可能。如果情况是这样的话，那么可以通过为在兼容模式下处理的文档特别提供支持来同时适应两种约定。

`\ifcompatibility`

此开关在文档以 `\documentstyle` 而不是 `\documentclass` 开始时设置。可以为任何条件提供适当的代码，如下所示：

```
\ifcompatibility
  \code emulating LaTeX 2.09 behavior
\else
  \code suitable for LaTeX2e
\fi
```

### 6.4 字体命令

Some font and size commands are now defined by the document class rather than by the  $\text{\LaTeX}$  kernel. If you are upgrading a  $\text{\LaTeX}$  2.09 document style to a class that does not load one of the standard classes, then you will probably need to add definitions for these commands. 目前，部分字体命令与字号命令将由文档类给出而非  $\text{\LaTeX}$  内核。若需要将  $\text{\LaTeX}$  2.09 文档风格升级为不加载标准模板类的文档类，则很可能需要为相关指令添加定义。

```
\rm \sf \tt \bf \it \sl \sc
```

这些短格式字体选择命令均未在  $\text{\LaTeX 2}_{\epsilon}$  内核中定义。它们由所有标准类文件定义。

如果要在类文件中定义它们，有几种合理的方法可以实现。

一种可以采用的定义是：

```
\newcommand{\rm}{\rmfamily}  
...  
\newcommand{\sc}{\scshape}
```

该实现可以使字体命令相互独立并共同作用；例如 `{\bf\it text}` 会生成粗斜体，如下所示：*text*。它还将使它们在数学模式下产生错误。

另一种可以采用的定义是：

```
\DeclareOldFontCommand{\rm}{\rmfamily}{\mathrm}  
...  
\DeclareOldFontCommand{\sc}{\scshape}{\mathsc}
```

This will make `\rm` act like `\rmfamily` in text mode (see above) and it will make `\rm` select the `\mathrm` math alphabet in math mode.

Thus `${\rm math} = X + 1$` will produce ‘ $\text{math} = X + 1$ ’.

If you do not want font selection to be orthogonal then you can follow the standard classes and define: 该实现可以使 `\rm` 在文本模式中表现得像 `\rmfamily` 一样（参见上文），并且它将使 `\rm` 在数学模式中选择 `\mathrm` 数学字母。

因此，`${\rm math} = X + 1$` 的实际效果是 ‘ $\text{math} = X + 1$ ’。

如果您不希望字体选择是正交的，那么可以遵循标准类并定义：

```
\DeclareOldFontCommand{\rm}{\normalfont\rmfamily}{\mathrm}  
...  
\DeclareOldFontCommand{\sc}{\normalfont\scshape}{\mathsc}
```

这意味着，例如，`{\bf\it text}` 将产生中等粗细（而不是粗体）的斜体，如下所示：*text*。

|                                                       |
|-------------------------------------------------------|
| <code>\normalsize</code><br><code>\@normalsize</code> |
|-------------------------------------------------------|

The command `\@normalsize` is retained for compatibility with L<sup>A</sup>T<sub>E</sub>X 2.09 packages which may have used its value; but redefining it in a class file will have no effect since it is always reset to have the same meaning as `\normalsize`. 命令 `@normalsize` 保留为与可能使用其值的 L<sup>A</sup>T<sub>E</sub>X 2.09 包兼容性，但在模板类文件中重新定义它将不起作用，因为它总是被重置为 `\normalsize` 相同的含义。

这说明现在**必须**在模板类内重定义 `\normalsize` 而非重定义 `—@normalsize—`；例如（一个相当不完整的重定义）：

```
\renewcommand{\normalsize}{\fontsize{10}{12}\selectfont}
```

请注意，—— 被 L<sup>A</sup>T<sub>E</sub>X 内核定义为一个错误消息。

|                                                                                          |
|------------------------------------------------------------------------------------------|
| <code>\tiny \footnotesize \small \large</code><br><code>\Large \LARGE \huge \Huge</code> |
|------------------------------------------------------------------------------------------|

这些其他的“标准”字号命令也都没有在内核中定义：如果需要，每个都需要在模板类文件中定义。而这些字号均在标准类中有定义。

这说明您应该对 `\normalsize` 使用 `\renewcommand`，对其他字体大小命令使用 `\newcommand`。

## 6.5 弃用命令

某些宏包可能无法在 L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub> 环境中运行，通常是因为它们依赖于一个从未得到支持并且现在已更改或已删除的 L<sup>A</sup>T<sub>E</sub>X 内部命令。

在许多情况下，现在可能会有一个健壮的、高级的方法来实现以前需要低级命令来完成任务。该部分请参阅第 4 节，看看现在是否可以使用 L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub> 模板类和宏包编写者的命令。

另外，当然如果您的宏包或文档类重新定义了任何内核指令（即在文件 `latex.tex`、`slitex.tex`、`lfonts.tex`、`sfonts.tex` 中定义的命令），那么您需要非常仔细地检查它，以确保实现是否已更改或者该命令在  $\text{\LaTeX}2_{\epsilon}$  内核中是否已不再存在。

$\text{\LaTeX}$  2.09 中的太多内部命令在  $\text{\LaTeX}2_{\epsilon}$  中已经重新实现或删除，在此无法列出所有的这些命令。您必须检查您使用或更改的所有命令。

然而，此处将列出一部分不再受支持的重要命令。

```
\tenrm \elvrm \twlrm ...
\tenbf \elvbf \twlbf ...
\tensf \elvsf \twlsf ...
:
```

这些形式的约七十个内部命令目前已经弃用。如果您的文档类或宏包使用了它们，请务必将它们替换为《 *$\text{\LaTeX}2_{\epsilon}$  Font Selection*》中描述的新字体命令。

例如，命令 `\twlsf` 应替换为：

```
\fontsize{12}{14}\normalfont\sffamily\selectfont
```

不兼容的另一种可能是使用在《 *$\text{\LaTeX}2_{\epsilon}$  for Authors*》中描述的 `rawfonts` 宏包。

另外，请记住，许多由  $\text{\LaTeX}$  2.09 预加载的字体现在已不再预加载。

```
\vpt \vipt \viipt ...
```

这些是  $\text{\LaTeX}$  2.09 中的内部大小选择命令。（它们仍然可以在  $\text{\LaTeX}$  2.09 兼容模式下使用。）请使用命令 `\fontsize` 替代：详见《 *$\text{\LaTeX}2_{\epsilon}$  Font Selection*》获取详情。

例如，`\vpt` 应替换为：

```
\fontsize{5}{6}\normalfont\selectfont
```

`\prm, \pbf, \ppounds, \pLaTeX ...`

L<sup>A</sup>T<sub>E</sub>X 2.09 使用了一些以 `\p` 开头的命令来提供“受保护”的命令。例如，`\LaTeX` 被定义为 `\protect\pLaTeX`，而 `\pLaTeX` 被定义为产生 L<sup>A</sup>T<sub>E</sub>X 标志。这使得 `\LaTeX` 变得健壮，尽管 `\pLaTeX` 并不是。

这些命令现已用 `\DeclareRobustCommand`（在第 4.10 节描述）重新实现。如果您的宏包重新定义了其中一个 `\p` 命令，那么您必须删除该重定义，并使用 `\DeclareRobustCommand` 重定义一个不含 `\p` 的命令。

`\footheight  
\@maxsep  
\@dblmaxsep`

这些参数在 L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub> 中未被使用，因此它们已被删除，除了在 L<sup>A</sup>T<sub>E</sub>X 2.09 兼容模式下。文档类不应再设置它们。

## 参考文献

- [1] Donald E. Knuth. *The T<sub>E</sub>Xbook*. Addison-Wesley, Reading, Massachusetts, 1986. Revised to cover T<sub>E</sub>X3, 1991.
- [2] Leslie Lamport. *L<sup>A</sup>T<sub>E</sub>X: A Document Preparation System*. Addison-Wesley, Reading, Massachusetts, second edition, 1994.
- [3] Frank Mittelbach and Michel Goossens. *The L<sup>A</sup>T<sub>E</sub>X Companion second edition*. With Johannes Braams, David Carlisle, and Chris Rowley. Addison-Wesley, Reading, Massachusetts, 2004.



## L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub> 总结：升级旧风格

本节包含对《*L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub> for Class and Package Writers*》的引用。

1. 应该编写一个模板类还是一个宏包？参见第 2.3 节，了解如何回答这个问题。
2. 如果使用了另一个样式文件，则需要获取该样式文件的更新版本。查看第 2.7.1 节，了解如何加载其他类和宏包文件的信息。
3. 测试一下：参见第 6.1 节。
4. 成功了吗？太好了，但是可能还有一些事情需要改变，以使您的文件成为一个结构良好、稳健和可移植的 L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub> 文件。因此，您现在应该阅读第 2 节，特别是 2.7 节。您还可以在第 3 节中找到一些有用的示例。

如果您的文件设置了新的字体、字体变化命令或符号，则还应该阅读《*L<sup>A</sup>T<sub>E</sub>X 2<sub>ε</sub> Font Selection*》。

5. 没有成功？这里有三种可能性：
  - 在读取您的文件时产生了错误消息；
  - 在处理测试文档时产生了错误消息；
  - 没有错误，但输出不符合预期。

不要忘记仔细检查最后一种可能性。

如果您已经到了这个阶段，那么您需要阅读第 6 节找到使您的文件工作的解决方案。